■ AI Practical Exam

Complete Questions, Answers & Code Explanations

Laboratory Practice-II — Artificial Intelligence

#	Assignment	Topic
1	DFS & BFS	Graph Traversal Algorithms
2	A* Algorithm	8-Puzzle Game Search Problem
3	Prim's MST	Greedy Algorithm
4	N-Queens	Backtracking & Branch and Bound
5	Chatbot	Customer Interaction Bot
6	Expert System	Information Management

Assignment 1: DFS & BFS (Graph Traversal)

■ THEORY QUESTIONS & ANSWERS

Q1. What are uninformed search algorithms?

These are search algorithms that have NO extra information about the goal. They only know the start and goal — nothing else. They blindly explore all nodes.
Examples: DFS, BFS, Uniform Cost Search.
Also called Blind Search.

Q2. What is DFS (Depth First Search)?

DFS explores as DEEP as possible along one branch before going back (backtracking).
→ It uses a STACK (or recursion).
→ Time Complexity: O(n^d)
→ Space Complexity: O(n*d)
→ Complete: YES (if tree is finite)
→ Optimal: NO (does not guarantee shortest path)

Q3. What is BFS (Breadth First Search)?

BFS explores ALL neighbors at current level before going deeper.
→ It uses a QUEUE.
→ Time Complexity: O(n^d)
→ Space Complexity: O(n^d)
→ Complete: YES
→ Optimal: YES (finds shortest path)

Q4. What data structure does BFS use?

QUEUE — because we explore level by level (FIFO — First In First Out).

Q5. What data structure does DFS use?

STACK — because we go deep first (LIFO — Last In First Out). In recursive DFS, the call stack acts as the stack.

Q6. Compare DFS vs BFS?

DFS: Goes deep → Uses Stack → Not optimal → Less memory
BFS: Goes wide → Uses Queue → Optimal → More memory
Both are complete for finite graphs.

Q7. What is a fringe?

Fringe is a data structure storing all nodes that CAN be explored next. In BFS it's a Queue, in DFS it's a Stack.

■ CODE EXPLANATION & CODE QUESTIONS

Simple Code Explanation:

Graph class → stores the graph as adjacency list (dictionary of lists).
add_edge() → adds a directed edge from source to destination.
dfs() → marks node visited, adds to result, then recursively visits unvisited neighbors.
bfs() → uses a deque (queue), marks start visited, pops node, visits unvisited neighbors.
main() → creates graph with 4 nodes, 6 edges, runs DFS and BFS from vertex 2.

OUTPUT:

```
DFS from vertex 2 → 2, 0, 1, 3 BFS from vertex 2 → 2, 0, 3, 1
```

Q1. Why is deque used instead of a list for BFS?

deque.popleft() is O(1) — very fast. list.pop(0) is O(n) — slow because all elements shift. So deque is more efficient for queue operations.

Q2. What happens if we don't mark nodes as visited?

Infinite loop! The algorithm will keep revisiting the same nodes forever and never stop.

Q3. How is adjacency list built in the code?

self.adj_list = {i: [] for i in range(vertices)} creates a dictionary.
add_edge(source, dest) appends dest to adj_list[source].

Q4. Trace BFS from vertex 2 for the given graph.

Start: Queue=[2], Visited=[2]
Pop 2 → neighbors: 0,3 → Queue=[0,3]
Pop 0 → neighbors: 1,2 (2 visited) → Queue=[3,1]
Pop 3 → no unvisited neighbors → Queue=[1]
Pop 1 → no unvisited → Queue=[]
Result: 2, 0, 3, 1

Q5. Why is DFS not optimal?

DFS may find a longer path first and return it. It doesn't guarantee the shortest path unlike BFS.

Assignment 2: A* Algorithm (8-Puzzle)

■ THEORY QUESTIONS & ANSWERS

Q1. What is the A* Algorithm?

A* is an advanced BFS that finds the SHORTEST path efficiently.
It uses a formula: f(n) = g(n) + h(n)
→ g(n) = actual cost from start to current node
→ h(n) = estimated cost from current to goal (heuristic)
→ f(n) = total estimated cost
It always picks the node with LOWEST f(n) first.

Q2. What heuristic is used in 8-Puzzle?

Manhattan Distance — sum of how many steps each tile needs to reach its goal position (moving only up/down/left/right).
Example: Tile at (0,0) but should be at (1,2) → Manhattan distance = |0-1| + |0-2| = 3

Q3. What are Open List and Closed List?

Open List → nodes discovered but NOT yet explored (like a to-do list).
Closed List → nodes already explored (already done).
A* picks the best node from open list each time.

Q4. Is A* complete and optimal?

YES to both — A* WILL find a solution if one exists, AND it will find the SHORTEST path, as long as the heuristic never overestimates (admissible heuristic).

Q5. What is an admissible heuristic?

A heuristic that NEVER overestimates the actual cost to reach the goal. Manhattan distance is admissible for 8-puzzle.

Q6. What are informed search algorithms?

Search algorithms that use EXTRA information (heuristics) to guide the search.
Examples: A*, Best First Search, Greedy Best First Search.
They are smarter and faster than uninformed algorithms.

Q7. List applications of A*.

1. GPS navigation / Google Maps
2. Game AI (pathfinding for characters)
3. Robot navigation
4. Puzzle solving (8-puzzle, 15-puzzle)

■ CODE EXPLANATION & CODE QUESTIONS

Simple Code Explanation:

PuzzleNode class → stores each puzzle state with g (moves taken), h (Manhattan distance), f=g+h, parent node, and move direction.
__lt__ → lets Python compare nodes by f value (needed for heap sorting).
manhattan_distance() → calculates total distance all tiles are from their goal positions.
find_blank() → finds where the empty space (0) is.
generate_neighbors() → tries moving blank Up/Down/Left/Right and returns valid new states.
a_star() → main function: uses min-heap (priority queue), always explores lowest f node first. Stops when goal reached.
reconstruct_path() → traces back from goal node to start using parent pointers.

OUTPUT: Total moves required = 3 (Right → Down → Right)

Q1. Why is heapq (min-heap) used?

heapq always gives the node with LOWEST f value first. This is the core of A* — always explore the most promising node next. Regular list would be O(n) to find minimum; heap is O(log n).

Q2. What does __lt__ do in PuzzleNode?

It defines the 'less than' comparison between two nodes. heapq needs to compare nodes to sort them, so __lt__ tells it to compare by f value (f = g + h).

Q3. What is best_cost dictionary used for?

It stores the best (lowest) cost found to reach each state. If we find a better path to a state, we update it. This prevents re-exploring states unnecessarily.

Q4. Calculate h-score for this start state: 1 2 3 / 0 4 6 / 7 5 8 → Goal: 1 2 3 / 4 5 6 / 7 8 0

Tile 4: at (1,1), goal (1,0) → distance=1
Tile 6: at (1,2), goal (1,2) → distance=0
Tile 5: at (2,1), goal (1,1) → distance=1
Tile 8: at (2,2), goal (2,1) → distance=1
Total h = 3

Q5. What does generate_neighbors() return?

It returns a list of (new_state, move_direction) tuples — all valid states reachable by moving the blank space in 4 directions.

Assignment 3: Prim's MST (Greedy Algorithm)

■ THEORY QUESTIONS & ANSWERS

Q1. What is the Greedy Algorithm?

Greedy algorithm builds a solution step by step, ALWAYS choosing the best available option at each step (locally optimal choice).
It is fast and simple but doesn't always give the globally optimal answer.
Exception: MST algorithms (Prim's, Kruskal's) DO give globally optimal results.

Q2. What is a Minimum Spanning Tree (MST)?

A spanning tree that connects ALL vertices of a graph with MINIMUM total edge weight and NO cycles.
For n vertices → MST has exactly n-1 edges.

Q3. Explain Prim's Algorithm step by step.

1. Start from any vertex (say 0).
2. Add it to the MST.
3. Look at all edges going OUT from MST vertices.
4. Pick the CHEAPEST edge that connects to a new (unvisited) vertex.
5. Add that vertex to MST.
6. Repeat steps 3-5 until all vertices are included.

Q4. Difference between Prim's and Kruskal's?

Prim's: Starts from a vertex, grows the MST one vertex at a time. Good for dense graphs.
Kruskal's: Sorts ALL edges first, picks cheapest edges one by one (avoiding cycles). Good for sparse graphs.

Q5. What are the 5 components of Greedy Algorithm?

1. Candidate Set — all available choices
2. Selection Function — picks best candidate
3. Feasibility Function — checks if candidate can be used
4. Objective Function — measures quality of solution
5. Solution Function — checks if solution is complete

Q6. What is Selection Sort?

A simple sorting algorithm:
→ Find minimum in unsorted part
→ Swap it with leftmost unsorted element
→ Repeat for remaining unsorted part
Time Complexity: $O(n^2)$ — not good for large data.

■ CODE EXPLANATION & CODE QUESTIONS

Simple Code Explanation:

Graph class → stores graph as adjacency list of (weight, neighbor) tuples.
add_edge(u,v,w) → adds edge BOTH ways (undirected graph).
prim_mst() → uses min-heap. Starts from vertex 0. Always picks cheapest edge to unvisited vertex.
visited[] → tracks which vertices are already in MST.
min_heap → stores (weight, current_vertex, parent_vertex). Heap ensures cheapest edge is always on top.
mst_edges → stores the edges selected for MST.
total_cost → sum of all selected edge weights.

OUTPUT: MST edges: 0-1(2), 1-2(3), 1-4(5), 0-3(6) | Total cost = 16

Q1. Why is a min-heap used in Prim's?

Min-heap always gives the CHEAPEST edge at top. So we always pick the minimum weight edge efficiently in O(log n) time instead of scanning all edges O(n).

Q2. What does visited[] array do?

It keeps track of vertices already added to MST. If visited[v] is True, we skip that vertex even if we encounter it again in the heap.

Q3. What happens when visited[current] is True?

We skip it using 'continue'. This is because a shorter path to this vertex was already processed earlier.

Q4. Trace first 2 steps of Prim's on the given graph (5 vertices).

Step 1: Start at 0. Heap=[(0,0,-1)]. Pop (0,0). Add neighbors: (2,1,0),(6,3,0) to heap.
Step 2: Pop (2,1,0) → cheapest edge. Add vertex 1 to MST. Edge 0--1=2 selected. Add neighbors of 1 to heap.

Q5. Why are edges added to adj_list in BOTH directions?

Because the graph is UNDIRECTED — if there's an edge between A and B, you can travel both A→B and B→A. So we add both directions.

Assignment 4: N-Queens (Backtracking & Branch and Bound)

■ THEORY QUESTIONS & ANSWERS

Q1. What is a Constraint Satisfaction Problem (CSP)?

A problem where we must find values for variables that satisfy all given rules/constraints.
Three components:
→ X = set of variables (e.g., queen positions)
→ D = domain of each variable (e.g., row numbers 1 to n)
→ C = constraints (no two queens attack each other)

Q2. What is Backtracking?

Try one option → if it fails, go back and try another option.
Like solving a maze: if you hit a dead end, come back and take a different turn.
It uses RECURSION.
Steps: 1.Place queen 2.Check if safe 3.If not safe → remove queen and try next row

Q3. What is the N-Queens problem?

Place N chess queens on an N×N board such that:
→ No two queens are in the same ROW
→ No two queens are in the same COLUMN
→ No two queens are on the same DIAGONAL
For n=4: there are exactly 2 solutions.

Q4. What is Branch and Bound?

It's an IMPROVED backtracking.
→ Uses boolean arrays to instantly check if a row/diagonal is under attack.
→ Faster than basic backtracking because checking is O(1) instead of O(n).
→ 'Bound' means we cut off (prune) branches that can't lead to a solution.

Q5. Difference between Backtracking and Branch & Bound?

Backtracking: Checks safety by scanning previous queens (slower O(n)).
Branch & Bound: Uses pre-computed arrays rows[], upper_diagonal[], lower_diagonal[] for instant O(1) checks. Much faster!

Q6. What is Graph Coloring?

Assign colors to graph vertices such that NO two adjacent (connected) vertices have the same color.
Chromatic number = minimum colors needed.
This is also a CSP problem.

■ CODE EXPLANATION & CODE QUESTIONS

Simple Code Explanation:

print_board() → prints Q for queen, dot(.) for empty cell.
is_safe(row, col) → checks 3 things: same row left, upper-left diagonal, lower-left diagonal.
backtrack(col) → tries placing queen in each row of current column. If safe, places it and recurses to next column. If stuck, removes queen (backtrack).
branch_and_bound(col) → same logic but uses 3 boolean arrays for O(1) safety checks:
→ rows[] = which rows are occupied
→ upper_diagonal[] = indexed by (row+col)
→ lower_diagonal[] = indexed by (n-1+col-row)
main() → runs both methods for n=4 and prints all solutions.

OUTPUT: 2 solutions found by both methods. Total solutions = 2.

Q1. How does is_safe() check diagonals?

Upper-left diagonal: move up-left (i--, j--) while checking board[i][j]==1
Lower-left diagonal: move down-left (i++, j--) while checking board[i][j]==1
If any queen is found → not safe.

Q2. What do upper_diagonal[] and lower_diagonal[] represent?

upper_diagonal[row+col] → all cells on same upper diagonal have same (row+col) value.
lower_diagonal[n-1+col-row] → all cells on same lower diagonal have same (n-1+col-row) value.
So checking these arrays tells us instantly if a diagonal is occupied.

Q3. Why is board[row][col] = 0 done after recursive call?

This is the BACKTRACKING step. If placing a queen here didn't lead to a solution, we REMOVE it (set to 0) and try the next row. Without this, the board would remain incorrectly filled.

Q4. For n=4, how many solutions exist and what are they?

2 solutions:
Solution 1: . Q . . / . . . Q / Q . . . / . . Q .
Solution 2: . . Q . / Q . . . / . . . Q / . Q . .

Q5. Why can't two queens be in same column?

In chess, queens attack vertically (same column). So if two queens share a column, they attack each other, which violates the problem constraint.

Assignment 5: Elementary Chatbot

■ THEORY QUESTIONS & ANSWERS

Q1. What is an AI Chatbot?

A chatbot is a computer program that SIMULATES human conversation.
→ Text-based chatbots: used on websites, social media for customer support.
→ Voice-based chatbots: used in call centers, smartphones (Siri, Alexa).
It understands user input and gives relevant responses.

Q2. How does a rule-based chatbot work?

It has a set of IF-ELSE rules:
→ If user says 'hello' → respond with greeting
→ If user says 'price' → respond with pricing info
Simple, fast, but limited — can only answer predefined questions.

Q3. How is it different from AI/ML chatbot?

Rule-based: Uses hard-coded IF-ELSE rules. Simple and predictable.
AI/ML chatbot: Learns from data using machine learning. Can understand context and new questions.
Our chatbot is RULE-BASED.

Q4. What are applications of chatbots?

1. Customer support on websites
2. Banking: check balance, transactions
3. Healthcare: symptom checker
4. E-commerce: order tracking
5. Education: doubt solving
6. Smartphones: Siri, Google Assistant

Q5. What are limitations of this chatbot?

1. Can only answer predefined questions
2. Cannot understand context or follow-up questions
3. Case-sensitive issues (solved by .lower())
4. Cannot learn from new conversations
5. No NLP — just keyword matching

■ CODE EXPLANATION & CODE QUESTIONS

Simple Code Explanation:

get_response(user_input) → main logic function:
→ Converts input to lowercase and strips spaces
→ Checks for keywords using 'in' operator
→ Returns matching response string
→ If no keyword matches → returns default help message

main() → runs infinite loop:
→ Takes user input
→ Calls get_response()
→ Prints bot reply
→ Breaks loop if user types 'exit', 'bye', 'quit', 'thanks'

OUTPUT TRACE: hi→greeting | laptops→products | orders→order info | exit→goodbye

Q1. Why is message.lower().strip() used?

.lower() converts all letters to lowercase so 'HI', 'Hi', 'hi' all match the same rule.
.strip() removes extra spaces before/after the input.
This makes the chatbot case-insensitive and input-clean.

Q2. How does get_response() match user input?

It uses keyword matching with 'in' operator.
Example: if 'price' in message → checks if word 'price' exists anywhere in user's message.
If match found → returns corresponding bot response.

Q3. What happens when user types 'exit'?

get_response() returns 'Bot: Thank you for visiting. Have a good day.'
Then in main(), the while loop checks if input is in exit words and BREAKS — ending the program.

Q4. How would you add a new topic — say 'discount'?

Add this in get_response():
if 'discount' in message:
return 'Bot: We offer 10% discount on orders above Rs.500.'
That's it! Just add one more if-condition.

Q5. Why use 'any(word in message for word in [...])' for greetings?

Because there are MULTIPLE greeting words: hello, hi, hey.
Instead of writing 3 separate if conditions, any() checks all of them at once and returns True if ANY one matches.

Assignment 6: Expert System (Information Management)

■ THEORY QUESTIONS & ANSWERS

Q1. What is an Expert System?

A computer system that SIMULATES the decision-making of a human expert.
→ Uses knowledge and rules from domain experts.
→ Can answer complex questions and give recommendations.
Example: Medical diagnosis system, Tax advisory system.

Q2. What are the 4 components of an Expert System?

1. Knowledge Base → Stores facts and rules from experts (like a database of knowledge).
2. Inference Engine → Applies rules to user input to reach a conclusion (the 'brain').
3. User Interface → How user interacts with the system (asks questions, gets answers).
4. Explanation Facility → Explains WHY the system gave a particular answer.

Q3. What is a Knowledge Base?

It's a collection of facts and rules provided by human experts.
In our code: KNOWLEDGE_BASE is a list of dictionaries.
Each entry has: name (system name), conditions (rules), explanation (justification).

Q4. What is an Inference Engine?

It's the core logic of the expert system.
→ Takes user input (facts)
→ Matches with rules in Knowledge Base
→ Finds best matching rule
→ Returns recommendation
In our code: infer_system() function is the inference engine.

Q5. What is the Explanation Facility?

It tells the user WHY a particular decision was made.
In our code: explain_decision() prints:
→ Recommended System name
→ Which rules matched
→ Justification text
This builds user TRUST in the system.

Q6. Give real-world applications of Expert Systems.

1. MYCIN — medical diagnosis (bacterial infections)
2. DENDRAL — chemical analysis
3. Tax advisory systems
4. Help desk systems
5. Stock market prediction
6. Fault diagnosis in machines

■ CODE EXPLANATION & CODE QUESTIONS

Simple Code Explanation:

KNOWLEDGE_BASE → List of 4 dictionaries. Each has 'name', 'conditions' (4 rules), and 'explanation'.

QUESTIONS → List of 4 questions to ask user: data_type, users, security, access.

ask_user() → Asks all 4 questions, stores answers in a dictionary called 'facts'.

infer_system(facts) → Compares user facts with each rule in KNOWLEDGE_BASE.
→ Counts how many conditions match (score)
→ Returns the rule with HIGHEST score as best match.

explain_decision() → Prints recommended system name, which rules matched, and justification.

main() → Calls ask_user() → infer_system() → prints result → explain_decision().

OUTPUT TRACE: data=student, users=few, security=medium, access=store → Office Document Management System (score=3)

Q1. What data structure is KNOWLEDGE_BASE?

It is a LIST of DICTIONARIES.
Each dictionary has keys: 'name', 'conditions', 'explanation'.
'conditions' is itself a dictionary with 4 key-value pairs.

Q2. How does infer_system() score rules?

For each rule in KNOWLEDGE_BASE, it counts how many conditions match the user's input.
Score = number of matching conditions (max 4).
The rule with HIGHEST score is returned as best recommendation.

Q3. Trace output for: data=student, users=few, security=medium, access=store

Library: data≠books(0), users≠many(0), security=medium(1), access≠search(0) → score=1
Hospital: data≠medical → score=1
Student: data=student(1), users≠many(0), security=medium(1), access≠update(0) → score=2
Office: data≠documents(0), users=few(1), security=medium(1), access=store(1) → score=3
WINNER: Office Document Management System

Q4. How would you add a new system to the knowledge base?

Add a new dictionary to KNOWLEDGE_BASE list:
{'name': 'Payroll System', 'conditions': {'data_type': 'salary', 'users': 'few', 'security': 'high', 'access': 'update'}, 'explanation': 'For salary data management.'}

Q5. What if two rules have equal score?

The code uses 'if score > best_score' (strict greater than).
So the FIRST rule with highest score wins.
If we want ties to be handled differently, we could use '>=' and keep all tied rules.

■ QUICK REVISION CHEAT SHEET

Assignment	Algorithm	Data Structure	Key Output
1 - DFS/BFS	DFS=Stack/Recursion BFS=Queue	Adjacency List (Dictionary)	DFS:2,0,1,3 BFS:2,0,3,1
2 - A*	f(n)=g(n)+h(n) Manhattan Distance	Min-Heap (heapq)	3 moves to solve 8-puzzle
3 - Prim's	Pick cheapest edge to unvisited vertex	Min-Heap + Adj List	MST Cost = 16 4 edges selected
4 - N-Queens	Backtracking=try+undo B&B=array checks	2D Board Array Boolean Arrays	2 solutions for n=4
5 - Chatbot	Keyword matching IF-ELSE rules	Dictionary (facts)	Rule-based customer support
6 - Expert Sys	Score-based matching Inference Engine	List of Dictionaries	Best matching rule wins

■ IMPORTANT POINTS TO REMEMBER

- ■ DFS = Stack = Goes DEEP first | BFS = Queue = Goes WIDE first
- ■ A* formula: f(n) = g(n) + h(n) | h = Manhattan Distance for 8-puzzle
- ■ Prim's: Always picks CHEAPEST edge to unvisited vertex using min-heap
- ■ N-Queens for n=4 has exactly 2 solutions | Backtracking = try + undo
- ■ Branch & Bound is FASTER than backtracking — uses O(1) diagonal checks
- ■ Chatbot uses .lower().strip() for case-insensitive keyword matching
- ■ Expert System: Knowledge Base + Inference Engine + UI + Explanation Facility
- ■ BFS is Optimal (shortest path) | DFS is NOT optimal
- ■ A* is both Complete AND Optimal with admissible heuristic
- ■ MST for n vertices has exactly n-1 edges and no cycles

Best of luck for your exam! ■